

PROJECT TESTING PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

This document presents the testing phase for ComicCraft, a web-based application that generates personalized comic stories, dialogues, illustrations, and downloadable PDF comics using FastAPI, Gemini models, and Stable Diffusion.

1. Testing Objectives

The main objective of testing is to verify that the complete ComicCraft workflow operates correctly from user input through AI generation, comic preview, image generation, layout construction, and PDF export. Testing also verifies that the FastAPI routes, frontend templates, and AI integration work together as intended.

- Verify that user inputs are correctly captured and processed.
- Verify generation of a structured 5-panel comic outline.
- Verify generation of narration and character dialogue.
- Verify Stable Diffusion image generation for each panel.
- Verify correct matching of images with their corresponding panel text.
- Verify that the comic preview displays all generated content correctly.
- Verify successful PDF creation and download.
- Verify the main FastAPI routes and JSON API workflow.
- Verify graceful handling of API, input, and generation errors.
- Verify that the local deployment can be started and accessed successfully.

2. Testing Scope

Testing covers the major components described in the project documentation: the HTML/CSS/Jinja2 frontend, FastAPI backend, Gemini Flash and Gemini Pro integration, Stable Diffusion image generation, comic layout construction, and FPDF-based PDF export.

3. Testing Environment

- Python with a project-specific virtual environment.
- FastAPI application served using Uvicorn.
- Jinja2 templates for frontend rendering.
- Google Gemini API for outline and story generation.
- Hugging Face Diffusers with Stable Diffusion for comic illustrations.
- FPDF for PDF generation.
- Web browser for frontend and generated-comic verification.
- FastAPI interactive API documentation at /docs for endpoint testing.

4. Functional Test Cases

Test ID	Feature	Test Input / Action	Expected Result	Status
TC-01	Homepage	Open /	index.html loads and displays the	To be tested

			comic input form.	
TC-02	User Input	Enter story prompt, character, setting, tone, and art style.	All values are accepted and submitted correctly.	To be tested
TC-03	Outline Generation	Submit a valid comic prompt.	A structured 5-panel outline is generated.	To be tested
TC-04	Story Generation	Use the generated outline.	Narration and character dialogue are generated for the panels.	To be tested
TC-05	Image Generation	Generate an image from each panel prompt.	Comic-style images are created and saved under static/panels.	To be tested
TC-06	Layout Builder	Pass generated images and story data.	Images and corresponding text are correctly paired in panel layout.	To be tested
TC-07	Comic Preview	Open the generated comic preview.	Panels show title, image, scene description, captions, and narration.	To be tested
TC-08	PDF Export	Click Download Your Comic as PDF.	A multi-page PDF is created and downloaded successfully.	To be tested
TC-09	Export Success	Complete PDF download.	Export success page is displayed.	To be tested
TC-10	JSON API	POST valid JSON to /generate-comic/json.	API returns generated layout data and PDF path.	To be tested
TC-11	Image Test Route	Send a prompt to /test-image.	An image is generated without creating a full comic.	To be tested
TC-12	Error Handling	Use invalid or incomplete input / simulate API failure.	Application handles the failure without crashing and reports an appropriate error.	To be tested

5. Integration Testing

Integration testing verifies that the separate ComicCraft modules operate correctly as a single pipeline.

- Frontend → FastAPI: Confirm form fields map correctly to backend Form(...) parameters.
- FastAPI → Gemini Flash: Confirm the story prompt produces a structured panel outline.
- Gemini Flash → Gemini Pro: Confirm the outline can be expanded into detailed narration and dialogue.

- Story/Image Prompts → Stable Diffusion: Confirm panel image prompts produce corresponding illustrations.
- Images + Story → Layout Builder: Confirm every generated image is paired with the correct panel text.
- Layout → FPDF Exporter: Confirm the complete layout is converted into a readable multi-page PDF.
- Export → Success Page: Confirm the user receives the expected export confirmation.

6. API and Route Testing

- / — verifies homepage rendering.
- /generate — verifies form-based comic generation.
- /generate-comic/json — verifies JSON-based comic generation and response data.
- /export-success — verifies successful export confirmation.
- /test-image — verifies image generation independently.

The FastAPI interactive documentation at /docs should be used to inspect and test the available API endpoints during local testing.

7. Frontend and UI Testing

- Check that all input labels and fields are visible and understandable.
- Check that story prompt, character name, setting, tone, and art style can be entered.
- Check button hover and interaction behavior.
- Verify the comic preview is readable and panels are clearly separated.
- Verify generated images, descriptions, captions, and narration are displayed in the correct order.
- Verify the PDF download control is visible and usable.
- Verify the export success page provides a clear confirmation and allows the user to create another comic.

8. AI Output Testing

AI output testing focuses on whether generated content follows the user's requested story prompt and preferences.

- Prompt adherence: generated story should reflect the supplied story idea.
- Character consistency: the requested main character should remain relevant throughout the panels.
- Setting consistency: scenes should reflect the requested location.
- Tone consistency: dramatic, funny, poetic, or light-hearted choices should influence the narrative.
- Art-style consistency: selected visual styles should be reflected in image prompts and generated images.
- Panel completeness: each panel should contain the required story and image information.
- Visual-text relationship: illustrations should correspond reasonably to their panel descriptions.

9. PDF Export Testing

- Confirm the PDF file is created in the configured static/exports location.
- Confirm each panel image is included.
- Confirm panel narration and story text are readable.
- Confirm panels appear in the correct sequence.

- Confirm the generated PDF can be opened by a standard PDF viewer.
- Confirm the exported file is available to the user after generation.

10. Negative and Error Testing

- Submit an empty story prompt.
- Submit incomplete form information.
- Send malformed JSON to the JSON generation route.
- Use unavailable or invalid API credentials.
- Test image generation with an invalid or unusable prompt.
- Test behavior when an AI service fails or times out.
- Test PDF generation when an expected image is missing.
- Verify exceptions are handled through the application's try-except and HTTPException mechanisms.

11. Performance Testing

Because ComicCraft performs multiple AI operations, performance testing should measure the time required for outline generation, story generation, image generation, full comic generation, and PDF export.

- Measure response time for /generate-comic/json.
- Measure total time required to generate all five panel images.
- Observe memory and CPU/GPU usage during Stable Diffusion generation.
- Check whether repeated generations remain stable.
- Verify that the application remains responsive during longer AI generation operations.

12. Usability Testing

- A first-time user should be able to understand the input form without technical knowledge.
- The flow from prompt submission to comic preview should be clear.
- Generated panels should be easy to read sequentially.
- The PDF download action should be obvious.
- The success page should clearly indicate that the comic was exported successfully.

13. Deployment Verification

After testing the individual modules, perform an end-to-end local deployment test using the documented Uvicorn command:

uvicorn app.main:app --reload

Open <http://127.0.0.1:8000> in a browser and verify the homepage. Then use /docs to verify the API endpoints and complete at least one full comic generation and export cycle.

14. Test Result Recording

Test ID	Result	Defect / Observation	Retest Result
TC-01	Pass / Fail / Blocked		
TC-02	Pass / Fail / Blocked		
TC-03	Pass / Fail / Blocked		

TC-04	Pass / Fail / Blocked		
TC-05	Pass / Fail / Blocked		
TC-06	Pass / Fail / Blocked		
TC-07	Pass / Fail / Blocked		
TC-08	Pass / Fail / Blocked		
TC-09	Pass / Fail / Blocked		
TC-10	Pass / Fail / Blocked		
TC-11	Pass / Fail / Blocked		
TC-12	Pass / Fail / Blocked		

15. Expected Testing Outcome

Successful completion of the testing phase should demonstrate that ComicCraft can accept personalized comic requirements, generate a structured storyline, create narration and dialogue, produce corresponding illustrations, display the completed comic through the web interface, and export the final comic as a downloadable PDF. The testing phase should also confirm that the documented FastAPI routes and local deployment workflow operate reliably.

16. Conclusion

The Project Testing Phase validates the end-to-end functionality of ComicCraft and provides a structured approach for identifying defects before wider deployment. Testing covers functional, integration, UI, AI-output, API, performance, error-handling, and PDF-export behavior. Once all critical test cases pass and identified defects are resolved and retested, the application can proceed toward future deployment and enhancement.